

SQUAREJUMP by Ethan Wu

SQUAREJUMP by Ethan Wu	1
Contact Details:	1
1. Introduction	1
2. Program Design	2
Main files used.....	2
How the game works	2
Gameplay design	2
Simple logic of the gameplay	2
3. Rules of the Game and How to Play.....	2
Objective.....	2
Rules	2
Controls.....	2
Difficulty	2
4. Implementation.....	2
Player movement	2
Obstacles	2
Collision detection.....	2
Score system.....	2
Screens.....	2
5. Development and Changes Made	2
6. Testing.....	2
7. Conclusion	2

Contact Details:

Names: Ethan Wu

Student Numbers: H00467645

Email: ecw4001@hw.ac.uk

1. Introduction

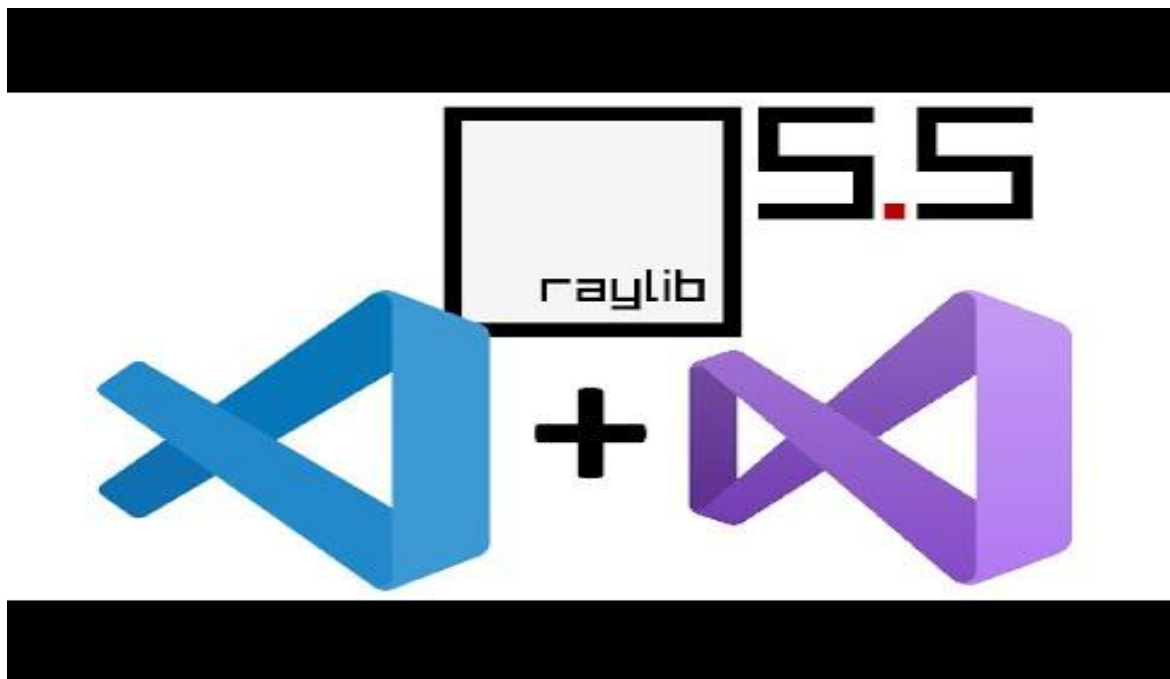
SQUAREJUMP is a simple 2D endless runner game made in C using the raylib library. The idea of the game is inspired by the Chrome dinosaur game, where the player has to jump over obstacles for as long as possible without crashing. The game gets harder over time because the obstacles keep coming and the speed increases gradually

The aim of this project was to create a small but complete game using C and raylib. I wanted the game to be simple to understand, easy to control, and clear enough that I could explain how each part works. I also wanted to practise using functions, variables, arrays, collision detection, and multiple game screens

To help set up the project, I used a raylib screen template from YouTube and then changed the code to make my own game. I will include the video link in my GitHub repository for reference

Template used:

https://www.youtube.com/watch?v=CnRpkKh_fjE



2. Program Design

The program is split into different screens:

- Title screen
- Gameplay screen
- Ending screen

This makes the program easier to organise. Each screen has its own .c file, and the game changes from one screen to another depending on what the player does

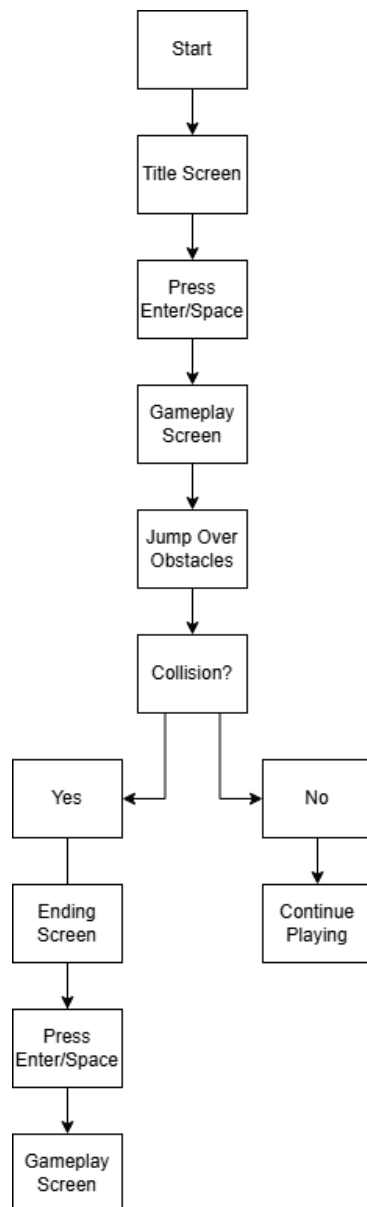
Main files used

- raylib_game.c - controls the overall game flow and screen switching
- screen_title.c - shows the title screen
- screen_gameplay.c - contains the main game logic
- screen_ending.c - shows the game over screen
- screens.h - shared function declarations

How the game works

When the program starts, it goes to the title screen. The player presses Enter or Space to begin. The game then switches to the gameplay screen. During gameplay, the square character can jump over obstacles. If the player hits an obstacle, the game ends and moves to the ending screen. The ending screen shows the score and high score, and the player can press a key to play again

Flowchart



Gameplay design

The gameplay screen contains the main parts of the game:

- **Player**
A black square that represents the character
Can jump using Space or Up Arrow
- **Obstacles**
Rectangles that move from right to left
The player must avoid them
- **Ground**
A line drawn across the screen to show where the player stands

- **Score**
Increases when the player successfully passes obstacles
- **High score**
Stores the best score reached during the session

Simple logic of the gameplay

The gameplay screen repeats this loop every frame:

1. Check if jump key is pressed
2. Apply gravity to the player
3. Move obstacles left
4. Reset obstacles when they leave the screen
5. Increase score when obstacles are passed
6. Check for collision
7. If collision happens, end the game

This is a simple and clear structure that makes the game easier to understand

3. Rules of the Game and How to Play

Objective

The goal of the game is to survive for as long as possible by jumping over obstacles. The longer the player survives, the higher the score

Rules

- The player starts on the title screen
- Press **Enter** or **Space** to start the game
- During the game, obstacles move towards the player from the right side of the screen
- The player must jump over the obstacles
- Each obstacle passed increases the score
- If the player touches an obstacle, the game ends
- The ending screen shows the final score and high score
- Press **Enter** or **Space** to play again

Controls

- **Enter** - start game / play again
- **Space** - jump
- **Up Arrow** - jump

Difficulty

The game becomes harder because the movement speed increases slowly over time. This means the player has less time to react as the game continues

4. Implementation

The game was programmed in C using raylib. Raylib made it easier to create a window, draw shapes, check keyboard input, and detect collisions

Player movement

The player uses a jump system based on vertical speed and gravity

- When the jump key is pressed, the player gets an upward speed
- Gravity then pulls the player back down
- When the player reaches the ground again, jumping stops

This makes the jump feel more natural than simply moving the square up and down by a fixed amount

Obstacles

The obstacles are stored using arrays for position, width, and height. This makes it possible to have more than one obstacle on screen. When an obstacle moves off the left side of the screen, it is reset to a new position on the right side.

This part of the design is useful because the game can keep running continuously without needing to create and destroy new objects all the time

Collision detection

Collision detection is done using rectangles. The player and each obstacle are treated as rectangular hitboxes. If the rectangles overlap, the game ends

This is a simple method but works well for this type of game

Score system

The score increases whenever the player passes an obstacle successfully. The high score updates if the current score is greater than the previous best score

Screens

Using separate screens made the program cleaner and easier to manage. Instead of putting everything in one file, I could keep each part of the game separate

5. Development and Changes Made

At the start, I used a raylib template to help create the basic screen system. This saved time because the title, gameplay, and ending screen structure was already set up. After that, I changed the code to create my own game idea

Some of the changes I made were:

- Changed the title screen to display **SQUAREJUMP**
- Created a simple jump system for the player
- Added moving obstacles
- Added collision
- Added scoring
- Added an ending screen showing score and high score
- Added replay controls

At one point the jump was too low, so it was not possible to get over the obstacles properly. I fixed this by increasing the jump speed and adjusting the gravity until the movement felt better

At another point the collisions were delayed so I updated and improved the detection

I kept the graphics simple by using basic shapes. This made the game easier to program and test

6. Testing

I tested the game by running it many times and checking:

- if the title screen appears correctly
- if the game starts when Enter or Space is pressed
- if the square jumps properly
- if obstacles move correctly
- if collisions are detected properly
- if the score increases after passing obstacles
- if the ending screen shows the correct score
- if the player can restart the game

I also tested whether the project could still build and run when copied into a clean folder with only the important source files and CMake files. This helped confirm that the submitted files are enough to rebuild the game.

7. Conclusion

In conclusion, I successfully created a simple endless runner game in C using raylib. The game includes a title screen, gameplay screen, and ending screen, along with player movement, obstacles, collision detection, score tracking, and repla

The final program is small but complete and shows the main ideas of game programming in C. I think the strongest parts of the game are that it is simple to understand, easy to play, and organised into separate files. If I had more time, I would improve the graphics, add sound effects, and maybe include more obstacle types to make the game more interesting

Overall, this project helped me understand how to structure a game program and how to use raylib to draw objects, read input, and manage game states